

Submitted By : Zeeshan

Python Bank ATM – Project Report

1. Project Overview (End-User Description)

Python Bank ATM is a console-based banking system designed to simulate the operations of a real ATM.

Key Features for Users:

1. Secure Login:

- Users log in using a card number and PIN.
- Up to 3 attempts allowed; account is blocked after 3 incorrect PIN entries.

2. Main Menu Options:

- **Balance Inquiry:** View your current account balance.
- **Deposit:** Add funds to your account with automatic two-decimal formatting.
- **Withdraw:** Withdraw funds respecting account type rules (Savings limit or Current overdraft).
- **Transaction History:** View all past transactions.
- **Exit:** End session safely, data is saved automatically.

3. Error Handling:

- Invalid PIN or card number is handled with clear messages.
- Amounts are validated for correct numeric entry.

4. Persistent Data Storage:

- Account details and transaction history are saved in a JSON file (accounts.json) for session persistence.
-

2. System Structure

File Organization:

File Name	Purpose
main.py	Entry point. Initializes accounts, launches ATM system.

File Name	Purpose
atm.py	Contains the ATM interface, menu system, login authentication, and exception handling.
account.py	Defines account classes, inheritance (SavingsAccount, CurrentAccount), encapsulation.
transaction.py	Optional: Handles transaction-specific operations (if implemented separately).
data/accounts.json	Stores account information and transaction history in JSON format for persistence.

Class Structure:

Account (Base Class)

└─ SavingsAccount (Overrides withdraw method)

└─ CurrentAccount (Overrides withdraw method)

ATM (User Interface & session management)

- **Inheritance:**

- SavingsAccount and CurrentAccount inherit from Account.
- Common methods like deposit(), get_balance(), get_transaction_history() are reused.

- **Polymorphism:**

- withdraw() is implemented differently for Savings and Current accounts.
- The ATM system calls current_user.withdraw(amount) without caring about the account type; behavior depends on subclass.

- **Encapsulation:**

- Private attributes: __balance, __pin, __transactions.
 - Accessed via getters and helper methods (get_balance(), _deduct(), _add_transaction()).
-

Technical Details:

1. **Programming Language:** Python 3.x

2. **Libraries Used:**

- getpass → hides PIN input securely.
- json → reading/writing persistent account data.

3. Error Handling:

- **Custom Exceptions:** CardNotFoundError and InvalidPINError.
- **Try/Except Blocks:** Prevent program crashes due to invalid inputs.

4. Data Persistence:

- Accounts and transaction history are saved in JSON format.
- Method to_dict() converts account objects to JSON-friendly dictionaries.

5. User Input Validation:

- PIN entry allows up to 3 attempts.
- Deposit/Withdrawal amounts are validated as positive numbers.

6. Formatting:

- All monetary values display **2 decimal places**.
- Main menu and messages are formatted for clarity and professional appearance.

Sample Interaction (End-User View):

```
=====
MAIN MENU
=====
```

```
1. Balance
2. Deposit
3. Withdraw
4. Transaction History
5. Exit
```

```
Choose: 1
```

```
Balance: 5000.00
```

- PIN authentication with 3 attempts:

```
Enter card number: 1111-2222
```

```
Enter PIN: ***
```

```
✗ Wrong PIN! Attempt 1 of 3
```

```
✗ Wrong PIN! Attempt 2 of 3
```

```
✗ Wrong PIN! Attempt 3 of 3
```

```
⊘ PIN entered incorrectly 3 times. Account blocked!
```

Bonus Features Implemented:

1. Persistent storage in JSON.
2. Two-decimal formatting for all monetary values.
3. Robust input validation for menu options and numeric amounts.
4. Professional, console-based UI formatting.
5. Secure PIN entry using `getpass`.

✅ Conclusion for End-User:

The Python Bank ATM provides a **secure, user-friendly, and realistic ATM experience**. All account operations are handled safely, and the system never crashes due to invalid input. Transaction history and balances are saved between sessions, ensuring consistency and reliability.